Layer Normalization

Atıl Samancıoğlu

1 Giriş: Batch Normalization Her Yerde Çalışır mı?

Bir önceki bölümde **Batch Normalization** öğrendik ve şunu gördük:

- Eğitim hızlanıyor
- Gradient'ler stabilize oluyor
- Learning rate seçimi kolaylaşıyor

Ama kritik bir soru var:

> *Batch Normalization her mimaride çalışır mı?*

Cevap: **Hayır.**
Özellikle:

- RNN
- LSTM
- Transformer

gibi modellerde BatchNorm ciddi problemler yaratır.
İşte bu noktada **Layer Normalization** devreye girer.

2 Batch Normalization Neden Yetersiz Kalır?

Batch Normalization şu varsayıma dayanır:

> *Aynı batch içindeki örnekler birlikte normalize edilebilir.*

Ama şu durumları düşün:

- Batch size = 1
- Değişken uzunluklu sequence'ler
- Online / streaming inference

Özellikle RNN ve NLP problemlerinde:

- Her örnek farklı uzunlukta
- Batch içi istatistikler anlamsız

> **BatchNorm batch'e bağımlıdır.**

Bu bağımlılık, sequence modellerinde büyük bir dezavantajdır.

3 Layer Normalization'ın Temel Fikri

Layer Normalization çok basit ama güçlü bir fikirle gelir:

> *Batch yerine, tek bir örneğin kendi özelliklerini normalize edelim.*

Yani:

- Batch boyunca değil
- Feature (neuron) boyunca normalize edilir

Bu sayede:

- Batch size önemli olmaz
- Her örnek bağımsız çalışır

4 Layer Normalization Nasıl Çalışır?

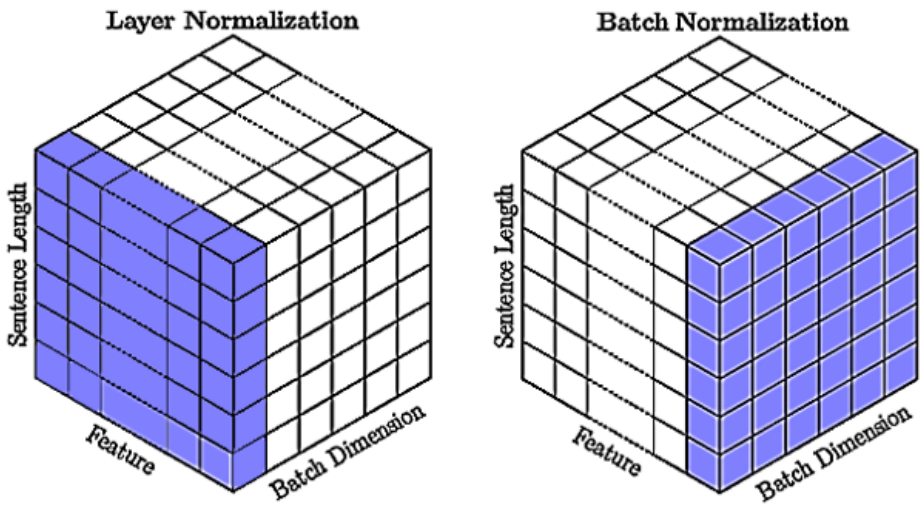

Figure 1: Layer Normalization

Tek bir örneği düşünelim:

$$\mathbf{h} = [h_1, h_2, \ldots, h_d]$$

Bu vektör bir katmanın çıktısıdır.

4.1 Adım 1: Ortalama

$$\mu = \frac{1}{d} \sum_{i=1}^{d} h_i$$

4.2 Adım 2: Varyans

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^{d} (h_i - \mu)^2$$

4.3 Adım 3: Normalize Et

$$\hat{h}_i = \frac{h_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

4.4 Adım 4: Ölçekle ve Kaydır

$$y_i = \gamma \hat{h}_i + \beta$$

Burada:

- Normalizasyon **tek örnek içinde**
- γ ve β öğrenilebilir

LayerNorm, her örneği kendi içinde dengeler.

5 BatchNorm vs LayerNorm

Özellik	BatchNorm	LayerNorm
Normalize edilen eksen	Batch	Feature
Batch size bağımlılığı	Var	Yok
RNN uyumu	Zayıf	Güçlü
Transformer uyumu	-	+
Inference stabilitesi	Batch'e bağlı	Stabil

6 Layer Normalization ve Transformer İlişkisi

Transformer mimarisi şunları içerir:

- Self-Attention
- Residual Connection
- Feed Forward Network

Ve her blokta şu yapı vardır:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Bu neden önemlidir?

1. Residual Connection

Residual bağlantılar sinyali korur ama:

- Aktivasyonlar büyüyebilir

LayerNorm:

- Bu büyümeyi dengeler

2. Self-Attention Dinamikleri

Self-Attention:

- Her token için farklı dağılım üretir

BatchNorm:

- Token'lar arası istatistik karışır

LayerNorm:

- Her token'ı ayrı normalize eder

Transformer = LayerNorm olmadan çalışmaz.

7 Dropout ile İlişkisi

Transformer'larda genellikle şu sıra kullanılır:

$$\text{Sublayer} \rightarrow \text{Dropout} \rightarrow \text{Residual} \rightarrow \text{LayerNorm}$$

Neden?

- Dropout gürültü ekler
- LayerNorm bu gürültüyü dengeler

Dropout bozar, LayerNorm toparlar.

Bu yüzden:

- LayerNorm + Dropout birlikte uyumludur
- BatchNorm + Dropout genelde sorunludur

8 Özet

- LayerNorm batch bağımsızdır
- Sequence modelleri için idealdir
- Transformer mimarisinin temel taşıdır
- Dropout ile birlikte güvenle kullanılır

BatchNorm CNN'lerin, LayerNorm Transformer'ların dilidir.